

Esercizio 4 - Creazione di un Team per un progetto

Indice

Indice.....	1
Introduzione.....	2
Soluzione A.....	2
Metodologia.....	2
Analisi dei costi.....	3
Soluzione B.....	3
Metodologia.....	3
Analisi dei costi.....	4
Formato di input e output con esempi.....	4

Introduzione

Dati C candidature e P partecipanti da selezionare tra questi, ognuno con un proprio valore di affidabilità e di criticità operativa, è necessario selezionare quelli per cui la differenza tra il livello di affidabilità più basso ed il livello di criticità operativa più alto risulta massima.

Soluzione A

Metodologia

In questa soluzione si adotta la metodologia dell'**albero decisionale con backtracking**: in questo caso per ogni candidatura si deve scegliere se **inserire il candidato** o meno nei **partecipanti**. Tramite questo metodo è stata creata una **funzione ricorsiva** *explore_decision_tree* che serve proprio ad esplorare l'**albero decisionale** dove ad ogni chiamata ricorsiva viene invocata la funzione *evaluate_current_team* per verificare se il **team corrente è migliore** in termini di qualità rispetto al team che fino a quel punto viene considerato il migliore, in caso lo fosse *best_team* viene aggiornato al team corrente. Al termine dell'esplorazione dell'albero quindi si ha la **certezza** che il team scelto sia il **migliore** di tutti.

Analisi dei costi

Trattandosi dell'esplorazione di un **albero decisionale completo con backtracking** la **complessità computazionale** si può ridurre infatti all'**esplorazione** di quest'ultimo, dato che l'albero decisionale è un **albero binario** il costo complessivo è:

$$O(2^C)$$

Per quanto riguarda invece la **complessità spaziale** questa è data principalmente dalla **dimensione** degli array *candidates*, *current_team*, *best_team* con un costo:

$$O(C) + O(P) + O(P) = O(C + P)$$

Ma siccome il **numero di partecipanti è sicuramente minore** del numero di candidature questo si traduce in:

$$O(C)$$

Complessità computazionale	$O(2^C)$	<i>All'esplorazione dell'albero decisionale</i>
Complessità spaziale	$O(C)$	<i>Dovuto all'array candidates</i>

Soluzione B

Metodologia

Invece di esplorare tutte le possibili combinazioni attraverso un **albero decisionale** questa soluzione utilizza un **max-heap** dividendo l'algoritmo in 2 fasi: Il primo passo consiste nell'**ordinare l'intero array** dei candidati in **ordine decrescente di affidabilità**, questo permette se si itera sull'array e si decide di includere nel team il candidato in posizione i-esima, selezionando i restanti P-1 membri esclusivamente tra i candidati esplorati in precedenza, di avere la certezza che **il candidato i-esimo sarà quello con l'affidabilità minima** all'interno del team; nel secondo passo, sapendo che l'affidabilità minima è fissata dal candidato i-esimo, per massimizzare la qualità del team è necessario che i restanti P-1 membri abbiano la **criticità massima più bassa possibile** e per ottenere questo risultato si utilizza proprio un **max-heap di dimensione massima P-1**

Analisi dei costi

Dalla descrizione precedente dell'algoritmo il **costo computazionale** è quindi legato all'**ordinamento** degli elementi tramite la funzione *qsort* che ha costo $O(C \log(C))$, a questo si aggiunge **per ogni candidato**, quindi eseguito in un ciclo *for* che si ripete C volte, **l'inserimento oppure la rimozione di un elemento del max-heap**. Siccome il max-heap è un albero **binario bilanciato** con altezza massima $\log(P)$ il costo per ogni singolo candidato è quindi $O(\log(P))$. Inoltre, dentro lo stesso ciclo, viene anche effettuata la **copia** degli id dei candidati dall'heap all'array *best_team* che ha costo $O(P)$. Di conseguenza il **costo computazionale** totale è:

$$O(C \log(C)) + O(C \log(P)) + O(C \cdot P) = O(C \log(C) + C \cdot P)$$

Analizzando la **complessità spaziale** invece è dovuta principalmente agli **array candidates** e *best_team* rispettivamente di **dimensione C e P** e alla **dimensione del max-heap che è P-1**, pertanto la **complessità spaziale** si può calcolare come segue:

$$O(C) + O(P) + O(P - 1) = O(C)$$

Questo perché, come detto in precedenza, **il numero P di partecipanti è sicuramente minore del numero C di candidature**

Complessità computazionale	$O(C \log(C) + C \cdot P)$	<i>Dovuto al sorting di candidates e all'aggiornamento di best_team</i>
-----------------------------------	----------------------------	---

Complessità spaziale	$O(C)$	<i>Dovuto alla dimensione di candidates</i>
-----------------------------	--------	---

Formato di input e output con esempi

Il formato di input e output è uguale per entrambe le soluzioni, con $n = 5$ si ha:

Input:

<C> <P>

Inserisci i dati per il candidato <i>: <affidabilità> <criticità>

4 2

Inserisci i dati per il candidato 1:

100 10

Inserisci i dati per il candidato 2:

90 20

Inserisci i dati per il candidato 3:

50 50

Inserisci i dati per il candidato 4:

40 60

Output:

La qualità massima ottenibile è: 70

Il team ottimo è composto dai candidati: 0 1